

Coding Standards Handbook

Version: 1.0

Last Updated: 05 July 2026



Prepared by



Table of Contents

Table of Contents	1
1. Introduction	4
1.1 Purpose	4
1.2 Scope	4
2. Technologies Used	5
2.1 Frontend	5
2.2 Backend	5
2.3 AI / Data Science	6
2.4 Data & Infrastructure	6
2.5 Testing & Observability	7
2.6 Tooling, CI/CD & Documentation	7
3. General Coding Principles	9
3.1 The Rule of One	9
3.2 Single Responsibility Principle (SRP)	10
3.3 LIFT	10
3.4 KISS	12
3.5 YAGNI	13
4. Repository Structure	14
4.1.1 Approved Directory Structure	15
4.1.2. Feature Organisation	15
4.1.3 Component and Service	16
4.2 Backend (Spring Boot)	17
4.2.1 Approved Directory Structure	18
4.2.2 Backend Rules	19
4.3 AI Service (Python)	20
4.3.1 Approved Directory Structure	21
4.3.2 AI Service Rules	21
4.4 Database (PostgreSQL)	22
4.4.1 Approved Folder Structure	22
4.4.2 Rules	22
4.4.3 Standard Database Workflow	23
4.4.4 Why do we use Flyway?	23
5. Naming Conventions	24
5.1 Purpose	24
6. Formatting Standards	26
6.1 Core Configuration Matrix	26

6.2 Team Automation Tools	27
6.3 Formatting Rules	27
6.4 Pre-Commit Checklist	27
7. Documentation and Comments	29
7.1 File Headers	29
7.2 Code Commenting Rules	30
7.3 In-Code Documentation Standards	31
7.4 Repository README Requirements	31
7.5 Pre-Commit Documentation Checklist	31
8. Technology-Specific Standards	33
8.1 Angular (Frontend)	33
8.2 Spring Boot (Backend)	35
8.3 Python AI Services	36
8.4 Database (PostgreSQL & Flyway)	37
8.5 Docker and AWS	38
9. Git, Version Control, and Issue Tracking	38
9.1 Core Workflow Principles	38
9.2 Commit Message Standard	38
9.3 Branching Strategy	40
9.4 Pull Request Validation	40
9.5 Issue and Task Tracking Rules	40
10. Error Handling and Input Validation	42
10.1 Core Principles	42
10.2 Backend (Spring Boot)	42
10.3 Frontend (Angular)	44
10.4 Python AI Service	44
10.5 Input Validation Rules	45
10.6 Pre-Commit Error Handling Checklist	45
11. Logging	47
11.1 Logging Levels	47
11.3 Python AI Service Implementation	48
11.4 Sensitive Data Restrictions	48
12. Security	50
12.2 Password Security	50
12.3 Input Validation	50
12.4 Database Safety	50
13. AI Usage Policy	51
Vibe Coding Policy	51

14. Testing Standards	52
14.1 The AAA Pattern	52
14.2 Test Naming Structure	53
14.3 Target Matrix by Framework Layer	53
15. Pre-Commit Verification Checklist	55
16. Code Review and Quality Control	56
16.1 Review Criteria Checklist	56
17. Definition of Done (DoD)	57
18. Development and Management Templates	59
18.1 Pull Request Template	59
18.2 Bug Report Template	60
18.3 Feature Request Template	60
18.4 Technical Task Template	61
19. Environment Setup and Project Onboarding	62
20. Compliance	63
21. References	64
Appendices	66
Appendix A: Issue and Project Management Templates	66
Appendix B: Conventional Commit Types	68
Appendix C: Environment Setup Checklist	69
Appendix D: Timesheet and AI Assistance Transparency Log	70
Appendix E: Glossary	71

1. Introduction

As a software project grows, it either becomes easier or harder to maintain. The deciding factor is not how smart the programmers are, but whether everyone consistently follows the same coding standards. This handbook exists to prevent technical debt(taking shortcuts which will later negatively impact the project and the team) by giving the team a shared set of rules based on professional, industry-recognized style guides for Angular, Spring Boot, Python, PostgreSQL etc. By following these standards, the code stays clean, understandable, and maintainable throughout the project instead of turning a promising system into something nobody wants to touch by the final demo.

1.1 Purpose

The purpose of this handbook is to:

- Reduce rework and technical debt by catching problematic/messy code early so we don't have to waste time fixing it later.
- Guarantee readability so that any team member can open any file and understand its purpose within minutes, without asking the author(the person who wrote the code).
- Make code review objective so reviews are based on agreed coding standards, not personal opinions.
- Protect the project's grade and demo-readiness by keeping the codebase stable, tested, and secure throughout the year.

1.2 Scope

These coding standards apply to every member of the team, every repository, and every part of the system. The rules must be followed from the moment a developer starts working on a feature until the application is deployed on AWS. If the team needs to make an exception, everyone must agree during a stand-up meeting, and that decision must be documented in **CONTRIBUTING.md** so it is transparent and consistent.

2. Technologies Used

Our team has agreed on a specific set of technologies for every part of the project. These technologies are the approved standards for the project. If someone wants to introduce a different library, framework, or service, the team must first discuss and approve it, and then update the handbook so everyone continues working with the same agreed technology stack.

2.1 Frontend

Technology	Purpose
Angular 17+	Frontend framework used to build the application's user interface as a Single Page Application (SPA).
Angular Material (Material UI)	Library of ready-made UI components such as buttons, forms, tables, and dialogs.
TypeScript	Programming language that adds strong typing to JavaScript to catch errors earlier.
SCSS	Styles the application's appearance using reusable and organised CSS features.
Tailwind CSS	CSS framework that provides utility classes for quickly creating layouts and spacing.

2.2 Backend

Technology	Purpose
Java 17+	Main programming language used to build the backend application.
Spring Boot 3.5+	Framework for building REST APIs and managing backend services.
Spring Security + JWT	Secures the application by handling user login (authentication) and permissions (authorization).

Spring Boot Actuator	Monitors the application's health, performance, and system information.
Spring WebClient	Allows the backend to communicate with other services, such as the Python AI service.
Spring Profiles	Uses different settings for development, testing, and production environments.
RESTful APIs	Standard way for the frontend, backend, and AI services to exchange data.
Swagger / OpenAPI	Automatically generates API documentation and makes APIs easier to test.

2.3 AI / Data Science

Technology	Purpose
Python 3.11+	Main programming language for AI, machine learning, and data processing.
PyTorch	Builds, trains, and runs deep learning models.
TensorFlow	Framework for building, training, and running machine learning models.
NumPy	Performs fast mathematical calculations and works with large datasets.
Scikit-learn	Builds and evaluates traditional machine learning models.
Matplotlib	Creates graphs and charts to visualise data and results.

2.4 Data & Infrastructure

Technology	Purpose
PostgreSQL	Stores and manages the application's data in a relational database.

Prisma	Connects the application to the database and manages database changes (schema migrations).
Redis	Speeds up the application by temporarily storing frequently used data (caching).
Docker & Docker Compose	Packages the application into containers so it runs consistently on any computer.
AWS (RDS, ECS, EC2, ElastiCache, S3, Parameter Store)	Cloud platform used to host the application, database, storage, and supporting services.

2.5 Testing & Observability

Technology	Purpose
JUnit 5 + Mockito + Spring Boot Test	Tests backend code to ensure it works correctly and reliably.
Jest	Tests Angular frontend components and functions.
Cypress	Tests the complete application by simulating real user interactions (End-to-End testing).
PyTest	Tests Python code and AI models.
Grafana + Prometheus	Monitors application performance and displays system metrics in dashboards.

2.6 Tooling, CI/CD & Documentation

Technology	Purpose
Git & GitHub	Tracks code changes (version control) and stores the project online for collaboration.
GitHub Actions	Automatically builds, tests, and deploys the application (CI/CD).

GitHub Projects	Organises tasks, issues, and sprint progress for the team.
Visual Studio Code	Recommended code editor (IDE) for developing the project.
Markdown	Simple formatting language used for README files and project documentation.
LaTeX	Creates professionally formatted academic reports and technical documents.

3. General Coding Principles

These rules apply to all technologies in the project, not just one. When a specific rule later in this handbook seems to conflict with a principle here, the principle wins (ask in the team channel).

3.1 The Rule of One

Each file should contain one main thing: one component, service, class, or module per file. A file that defines two Angular components, or a Java file with two public classes, has already failed a code review.

Good Example	Bad Example
<pre>StudentController.java Java public class StudentController { // Handles student-related requests } login.component.ts TypeScript export class LoginComponent { // Handles the login page }</pre>	<pre>StudentController.java Java public class StudentController { ... } public class LecturerController { ... }</pre>

3.2 Single Responsibility Principle (SRP)

Every class, function, or component does one job well. If you find yourself describing a function with "and" ("this validates the form and saves it and sends an email"), split it.

Good Example	Bad Example
<pre>public boolean validateUser(User user) { return user.getEmail() != null; } public void saveUser(User user) { userRepository.save(user); } public void sendWelcomeEmail(User user) { emailService.send(user); }</pre> Note: Each method has exactly one job.	<pre>public void registerUser(User user) { validateUser(user); userRepository.save(user); emailService.send(user); logger.info("User registered"); }</pre>

3.3 LIFT

- Locate: code should be easy to find by feature name.

○

Good Example:	Bad Example:
<pre>src/ authentication/ login/ register/</pre>	<pre>src/ misc/ stuff/ random/</pre> Problem: Nobody immediately

	knows where the login code is hidden.
--	---------------------------------------

- Identify: a filename should tell you what's inside initially.

○

<i>Good Example:</i>	<i>Bad Example:</i>
• StudentService.java • InvoiceGenerator.java • LoginComponent.ts	• temp.java • newfile.java • test2.ts • thing.py • Problem: These names do not explain what the file actually does.

- Flat: keep folder nesting as shallow as possible.

○

<i>Good Example:</i>	<i>Bad Example:</i>
<pre>src/ users/ UserService.java UserController.java</pre>	<pre>src/ application/ backend/ modules/ users/ services/ implementation/ current/ UserService.java</pre> Problem: Too many deep folders make navigation difficult and

	tedious.
--	----------

- Try to be DRY: Don't Repeat Yourself, but don't abstract prematurely (meaning do not create reusable code before you actually need it) either (see KISS).

Good Example:	Bad Example:
<pre>public double calculateVAT(double price) { return price * 0.15; }</pre> Usage across the application: <pre>double total = calculateVAT(price);</pre>	<pre>price1 * 0.15 price2 * 0.15 price3 * 0.15 price4 * 0.15</pre> Problem: The exact same hardcoded calculation is repeated multiple times.

3.4 KISS

Keep It Simple. Prefer the boring, obvious solution over a clever one.

Over-engineering (too many classes, too many design patterns, building for problems that don't exist yet etc) costs more in a student team.

Good Example:	Bad Example:
<pre>public boolean isAdult(int age) { return age >= 18; }</pre> Note: Clean and easy to understand.	<pre>public boolean isAdult(int age) { Boolean result = Boolean.valueOf(Integer.compare(age, 18) >= 0); return result.booleanValue(); }</pre> Problem: It works, but it is unnecessarily over-engineered

3.5 YAGNI

You Aren't Gonna Need It. Build exactly what the current sprint or assignment requires. Don't build features until they're actually needed.

4. Repository Structure

To maintain a clean, consistent, and easy-to-navigate codebase, all project repositories should follow a feature-based folder structure. Files that belong to the same feature must be stored together, supporting the LIFT (Locate, Identify, Flat) principle introduced in Chapter 3.

4.1 Frontend (Angular)

The main frontend folders have the following responsibilities:

Folder	Purpose	Example Files
core/	Contains code that is used throughout the whole application, such as authentication, route guards and global services.	<code>auth.service.ts</code> <code>auth.guard.ts</code> <code>jwt.interceptor.ts</code>
shared/	Contains reusable components, pipes, directives and styles that can be used by many features.	<code>button.component.ts</code> <code>header.component.ts</code> <code>footer.component.ts</code> <code>tatus-chip.component.ts</code>
features/	Contains the main features of the application. Each feature keeps its own components, services, styles and tests together.	<code>login/login.component.ts</code> <code>timesheets/timesheets.component.ts</code> <code>users/logtime/logtime.component.ts</code>
layouts/	Contains layouts that define the structure of pages, such as navigation bars, sidebars and footers.	<code>main-layout.component.ts</code> <code>dashboard-layout.component.ts</code>

		<code>_page-layouts.scss</code>
pages/	Contains standalone pages that are not part of a specific feature.	<code>not-found.component.ts</code> <code>log-time-placeholder.component.ts</code>

4.1.1 Approved Directory Structure

```

src/app/
├── core/
├── features/
│   ├── login/
│   ├── signup/
│   ├── timesheets/
│   └── users/
├── shared/
├── layouts/
├── pages/
├── app.config.ts
└── app.routes.ts

```

All new frontend features should follow this folder structure unless the team agrees on a documented exception.

4.1.2. Feature Organisation

All files related to the same feature must be stored in the same folder. Keeping related files together makes the project easier to understand, maintain and update.

Good Example	Bad Example
	features/

Everything related to the Timesheets feature is stored together.

features/

```
└─ timesheets/
    │   timesheets.component.ts
    │   timesheets.component.html
    │   timesheets.component.scss
    │
    └─ timesheets.component.spec.ts
        └─ timesheet.service.ts
```

```
└─ login/
```

```
│   └─ login.component.ts
```

```
shared/
```

```
│   └─ login.component.scss
```

```
pages/
```

```
│   └─ login.component.html
```

```
core/
```

```
│   └─ login.component.spec.ts
```

In this example, the files for one feature are spread across multiple folders. Developers have to search the project to find everything related to the Login feature, making the code harder to maintain.

4.1.3 Component and Service

Angular separates the user interface from the application's business logic.

- **Components:** Responsible for displaying information and responding to user actions, such as button clicks and form submissions.
- **Services:** Responsible for handling business logic and communicating with the Spring Boot backend using Angular's HttpClient.

Approved Data Flow

1. User Interface: Initiates action (e.g., click)
2. Angular Component: Handles the UI presentation
3. Angular Service: Contains business logic and manages API calls
4. Spring Boot REST API: Backend routing and request handling

5. PostgreSQL Database: Persistent data storage

Good Example	Bad Example
The Component asks the Service to retrieve data, keeping presentation completely separated from data fetching. <pre>timesheets.component.ts loadTimesheets() { this.timesheetService.getTimesheets(); } timesheet.service.ts getTimesheets() { return this.http.get("/api/timesheets"); }</pre>	The Component communicates with the backend directly. <pre>timesheets.component.ts loadTimesheets() { this.http.get("/api/timesheets"); }</pre> Why? This practice mixes user interface code with backend communication, making the component significantly harder to test, maintain, and reuse.

4.2 Backend (Spring Boot)

The main backend folders have the following responsibilities:

Folder	Purpose	Example Files
controller/	Receives HTTP requests and returns responses. Controllers should only handle communication between the frontend and backend.	<code>AuthController.java</code> , <code>TimeEntryController.java</code>

service/	Contains the application's business logic, validation and calculations.	<code>AuthService.java</code> , <code>TimerService.java</code>
repository/	Communicates with the PostgreSQL database using Spring Data JPA.	<code>UserRepository.java</code> , <code>ProjectRepository.java</code>
domain/	Contains the application's database entities (tables).	<code>User.java</code> , <code>Project.java</code>
dto/	Contains request and response objects used to transfer data between the frontend and backend.	<code>RegisterRequest.java</code> , <code>AuthResponse.java</code>
mapper/	Converts Entities into DTOs and DTOs into Entities.	<code>TimeEntryMapper.java</code>
config/	Stores application configuration such as security, Swagger and authentication.	<code>SecurityConfig.java</code> , <code>SwaggerConfig.java</code>
enums/	Stores constant values used throughout the application.	<code>UserStatus.java</code> , <code>EmploymentType.java</code>
exception/	Stores custom exceptions and global exception handling.	<code>ResourceNotFoundException.java</code>

4.2.1 Approved Directory Structure

```

src/
├── main/
│   ├── java/
│   │   └── timesheets/
│   │       ├── controller/
│   │       ├── service/
│   │       ├── repository/
│   │       ├── domain/
│   │       └── dto/

```

```
| | | | request/
| | | | response/
| | | | mapper/
| | | | enums/
| | | | config/
| | | | exception/
| | | resources/
| | | application.yml #Common configuration
| | | application-dev.yml #Local development settings
| | | application-staging.yml #Staging/testing environment
settings
| | | application-prod.yml #Production (AWS) settings
| | | db/
| | | migration/
| test/
| java/
| timesheets/
```

4.2.2 Backend Rules

Do

- Keep Controllers focused on receiving requests and returning responses.
- Place all business logic inside Services.
- Access the database only through Repositories.
- Use DTOs when sending or receiving data from the frontend.
- Use Constructor Injection for dependencies instead of field injection (`@Autowired`).
- Keep configuration classes inside the `config/` package.
- Keep helper methods inside the `mapper/` or `util/` package where appropriate.
- Mark injected dependencies as **final** whenever possible.

Avoid

- Putting business logic inside Controllers.
- Accessing the database directly from Controllers or Services without using a Repository.
- Returning Entity objects directly to the frontend.
- Using **field injection** (`@Autowired` directly on fields).
- Placing unrelated classes inside the same package.

Why?

Giving each folder one responsibility makes the backend easier to read, test, debug and maintain. Constructor Injection makes dependencies clear, allows fields to be immutable (`final`), and makes unit testing with Mockito much easier because Spring does not need to create the object during testing.

4.3 AI Service (Python)

The main AI folders have the following responsibilities:

Folder	Purpose	Example Files
api/	Receives requests from the Spring Boot backend.	<code>predict.py</code>
models/	Stores machine learning and deep learning models.	<code>productivity_model.py</code>
services/	Contains AI processing logic and prediction functions.	<code>prediction_service.py</code>
schemas/	Validates incoming and outgoing data using Pydantic.	<code>prediction_schema.py</code>
utils/	Contains reusable helper functions such as data preprocessing.	<code>preprocessing.py</code>
notebooks/	Stores Jupyter notebooks used for experiments and model training only.	<code>training.ipynb</code>
tests/	Contains unit tests for the AI service.	<code>test_prediction.py</code>

4.3.1 Approved Directory Structure

```
ai_service/  
├─ app/  
│   ├─ api/  
│   ├─ models/  
│   ├─ services/  
│   ├─ schemas/  
│   └─ utils/  
├─ notebooks/  
├─ tests/  
├─ requirements.txt  
└─ Dockerfile
```

4.3.2 AI Service Rules

Do

- Keep production code inside the app/ folder.
- Store AI models inside the models/ folder.
- Validate all incoming data using Pydantic Schemas.
- Place AI processing logic inside the services/ folder.
- Store experiments and research work inside the notebooks/ folder.

Avoid

- Importing notebook code into the production application.
- Placing AI models inside the api/ folder.
- Processing data before it has been validated.
- Mixing experimental code with production code.

Why?

Separating production code from experiments makes the AI service easier to maintain, test and deploy. It also reduces the chance of unfinished or experimental code being included in the final application.

4.4 Database (PostgreSQL)

Our project uses Flyway to manage all database changes systematically.

Folder	Purpose	Example Files
db/migration/	Stores all Flyway migration files used to create and update the database schema.	V1__init_schema.sql, V2__add_timesheet_table.sql

4.4.1 Approved Folder Structure

```
src/  
└─ main/  
    └─ resources/  
        └─ db/  
            └─ migration/  
                ├── V1__init_schema.sql  
                ├── V2__add_timesheet_table.sql  
                └─ V3__update_user_roles.sql
```

4.4.2 Rules

Do

- Create one migration file for every database change.
- Name migration files using the format:
`V<VersionNumber>__Description.sql`
- Flyway requires a **double underscore** (__)
- Commit migration files together with the feature that uses them.

Avoid

- Manually creating or editing tables in PostgreSQL.

- Editing an old migration after it has already been committed and run.
- Making database changes without creating a migration.

4.4.3 Standard Database Workflow

1. Create a new migration file.
2. Add the SQL statements needed for your database change.
3. Run the application locally. Flyway will automatically apply the new migration.
4. Check that the database was updated correctly.
5. Commit the migration together with your feature.

Test-Only Migrations

If database data is required for automated tests, migration scripts should be placed in:

```
src/test/resources/db/migration/
```

This keeps testing data separate from the production database.

4.4.4 Why do we use Flyway?

Using Flyway helps us to:

- Keep every developer's database the same.
- Automatically update the database when the application starts.
- Track database changes using Git.
- Make it easier to fix mistakes or roll back changes if needed.

5. Naming Conventions

5.1 Purpose

Consistent naming makes the code easier to read and understand. If everyone follows the same naming style, teammates can quickly find files, classes and methods without guessing.

Element	Naming Standard	Example	Why?
Angular/TypeScript Files	feature.type.ts	<code>login.component.ts</code> <code>auth.service.ts</code>	Makes it easy to identify the file's purpose.
Java, TypeScript & Python Classes	PascalCase	<code>TimesheetService</code> <code>UserProfileComponent</code>	Class names stand out from variables and methods.
TypeScript Interfaces	PascalCase (no I prefix)	<code>User</code> , <code>TimeEntry</code>	Matches modern TypeScript standards and keeps names simple.
Java & TypeScript Variables and Methods	camelCase	<code>userId</code> <code>calculateTotalHours()</code>	The standard naming style for Java and TypeScript.
Python Variables and Functions	snake_case	<code>user_id</code> <code>calculate_total_hours()</code>	Follows the official Python style guide (PEP 8).
Constants	UPPER_SNAKE_CASE	<code>MAX_LOGIN_ATTEMPTS</code>	Makes constants easy to recognise because they never change.

Database Tables	Plural snake_case	users timesheet_entries	Tables store many records, so plural names make sense.
Database Columns	Singular snake_case	user_id created_at	Each column stores one value for each record.
Angular Selectors	app-kebab-case	app-timesheet-card	Matches Angular's recommended naming convention.
REST API Endpoints	Plural, kebab-case and versioned	/api/v1/timesheet-entries	Keeps APIs consistent and easier to extend in future versions.
Angular Test Files	Same file name + .spec.ts	login.component.spec.ts	Makes it easy to find the test for a component or service.
Java Test Files	Same class name + Test	AuthServiceTest.java	Clearly links each test to the class being tested.
Python Test Files	test_ + module name	test_prediction_service.py	Follows the standard Python testing convention.
Git Branches	type/short-description	feature/login-page fix/login-token	Makes branches easy to identify and organise.

6. Formatting Standards

Consistent formatting ensures the codebase looks like it was written by a single person. This reduces friction during code reviews and keeps the focus entirely on code logic. Formatting is mandatory before committing code.

6.1 Core Configuration Matrix

Rule	Angular / TypeScript	Java (Spring Boot)	Python	SQL
Indentation	2 spaces	2 spaces	4 spaces	2 spaces
Max Line Length	100 chars	100 chars (120 max)	88 chars	100 chars
Braces {}	Same line	Same line	Indentation only	N/A
Quotes	Single ' (TS) / Double " (HTML)	Double "	Consistent ' or "	Single ' for text
Tabs vs Spaces	Spaces only	Spaces only	Spaces only	Spaces only
Formatter	Prettier	Spotless (Google Style)	Black + isort	sqlfluff
Linters	ESLint	Checkstyle	Ruff	sqlfluff

6.2 Team Automation Tools

Instead of manually fixing style issues, rely on the project's automated tooling pipeline:

- **Formatters (Arranges Layout):**
 - Prettier: Handles TypeScript, HTML, and SCSS files.
 - Spotless: Enforces the Google Java Style Guide on backend code.
 - Black + isort: Formats Python files and organizes import sorting.
 - sqlfluff: Automatically cleans up SQL script structures.
- **Linters (Finds Bugs and Bad Practices):**
 - ESLint: Catches logical errors and anti-patterns in frontend code.
 - Checkstyle: Validates Java standards adherence.
 - Ruff: Inspects Python code for bugs and security issues.

6.3 Formatting Rules

Do

- Enable Format on Save in your IDE (e.g., VS Code or IntelliJ) to automate this workflow.
- Run your language formatter before staging code for a Git commit.
- Keep code lines short enough to read without horizontal scrolling.
- Fix all compiler, linter, and formatter warnings locally.

Avoid

- Never use the Tab key if your editor is not configured to insert spaces instead.
- Never mix tabs and spaces in the same file, as it breaks cross-platform rendering.
- Never manually fight the formatter output, adjust the configuration files if necessary.

6.4 Pre-Commit Checklist

Before executing git commit, verify the following loop:

1. Format: Run the language-specific formatter tool.
2. Lint: Run the linter and fix any code quality alerts.
3. Verify: Ensure the codebase compiles successfully with zero style violations.

Why? Code that fails formatting or linting checks will automatically be rejected by our CI/CD deployment pipeline. Always check locally first :)

7. Documentation and Comments

Code should be clean enough to explain how it works, while comments and documentation exist to explain why it was built that way.

7.1 File Headers

Purpose

File headers help developers quickly understand what a file does, who created it, and which project requirement it implements. They also provide **requirements traceability**, making it easier to verify that every Functional Requirement (FR) has been implemented.

- Java & TypeScript Standard:

```
/**  
  
 * Author: Jane Doe  
 * Date: 2026-07-02  
 * Purpose: Handles CRUD operations for weekly timesheet  
entries.  
 * Related Requirement:FR-04 - Timesheet Submission  
 * The requirement ID must match the project's  
 * Requirement-Use Case Traceability Matrix (RUTM).  
 */
```

The Related Requirement must match the project's Requirement-Use Case Traceability Matrix (RUTM) to ensure every implemented feature can be traced back to an approved functional requirement.

- Python Standard (Module Docstring):

```
"""  
  
Handles AI predictions for employee productivity.  
  
Author: Jane Doe  
Date: 2026-07-02  
"""
```

Why?

File headers improve documentation, make files easier to understand, and provide **requirements traceability**.

Linking every major file to a Functional Requirement (FR) or Non-Functional Requirement (NFR) helps verify that every project requirement has been implemented and tested, preventing unnecessary features from being added.

7.2 Code Commenting Rules

Do

- Explain complex business rules or grading rubric edge cases.
- Document workarounds, temporary fixes, or third-party API limitations.
- Update your comments immediately if you change the underlying code.

Avoid

- Never comment every single line. It creates visual noise.
- Never state the obvious. (e.g., Avoid `// Increment x` followed by `x++`;).
- Never leave dead or commented-out code blocks in the repository. Use Git history if you need to find old code.

Good Example:	Bad Example:
---------------	--------------

```
// AWS limits requests to 5 per
second, so data is pushed in
batches.
```

```
// Check if user is null
if (user == null) {
```

7.3 In-Code Documentation Standards

Public-facing classes, methods, and API configurations must use the language-appropriate documentation format so IDEs can auto-generate tooltips:

Language	Standard	Application
Java	Javadoc (<code>/** ... */</code>)	Public classes, controllers, and service methods.
TypeScript	JSDoc (<code>/** ... */</code>)	Component properties (<code>@Input/@Output</code>) and core services.
Python	Docstrings (<code>""" ... """</code>)	Public modules, helper functions, and schemas.

7.4 Repository README Requirements

Every independent project folder or service root directory must contain a README.md file covering:

- Overview: A brief statement explaining what the service does.
- Local Setup: Precise step-by-step commands to build and boot the application locally.
- Testing: The command needed to execute the local test suites.
- Environment Variables: A list of required key names (e.g., DATABASE_URL), but never the actual passwords or secret values.
- Docker: Any commands required to spin up containerized dependencies (like a local database container).

7.5 Pre-Commit Documentation Checklist

Before pushing code to your team's repository, run through these four criteria:

1. Headers: Do new files contain authorship and purpose headers?

2. Context: Do the comments explain the logic's intent rather than just repeating the code syntax?
3. Cleanliness: Have all temporary comments and commented-out code lines been deleted?
4. Readme Sync: If your feature introduced a new environment variable or installation step, is the README.md updated?

8. Technology-Specific Standards

8.1 Angular (Frontend)

These standards keep the frontend organized, predictable, and easy to maintain as the project grows.

Architecture Rules

Do

- Use Standalone Components for all new components.
- Keep components focused on one task.
- Move large HTML and SCSS code into separate .html and .scss files.
- Split large components into smaller ones when they become difficult to manage.

Avoid

- Creating new NgModules unless the team has agreed there is a specific reason.
- Putting too much functionality into one component.

Why? Small components are easier to read, test, and reuse.

Services & State Rules

Do

- Use Angular Services to communicate with the Spring Boot backend.
- Keep API calls and business logic inside Services.

Avoid

- Calling HttpClient directly inside a Component.

Good Example:

```
// login.component.ts
this.authService.login(user);

// auth.service.ts
```

Bad Example:

```
// login.component.ts

this.http.post('/api/login',
user);
```

<pre>login(user: LoginRequest) { return this.http.post('/api/login', user); }</pre>	
---	--

Why? Keeping API calls inside Services makes components simpler and keeps all backend communication in one place.

Styling Rules

Do

- Use Angular Material components whenever possible.
- Use Tailwind CSS for layouts and spacing.
- Use SCSS for component-specific styles and reusable variables.

Avoid

- Creating custom components when Angular Material already provides one.

Design Token Rule

Developers must use the colours, spacing, typography, border radius and other design tokens defined in the team's **Brand Style Guide**.

Avoid creating custom values unless the design has been updated and approved.

Why?

Following the same design tokens keeps the user interface consistent and prevents visual differences ("design drift") across the application.

8.2 Spring Boot (Backend)

The backend follows a clear layered architecture so that every class has one responsibility: Controller to Service to Repository to Database. Each layer should only communicate with the layer directly below it.

Controllers

Do

- Handle HTTP requests and responses.
- Return the correct status codes.
- Call Services to perform business logic.

Avoid

- Writing business logic inside Controllers.Java

Good Example	Bad Example
<pre>@PostMapping("/users") public UserResponse createUser(@RequestBody UserRequest request) { return userService.createUser(request); }</pre>	<pre>@PostMapping("/users") public UserResponse createUser(...) { // Hundreds of lines of validation, calculations, and database queries }</pre>

Services & Repositories

Do

- Place business logic inside Services to handle calculations and validation.
- Use Repositories only for database operations.

Avoid

- Placing business logic inside Repository classes.

Why? Services keep Controllers small and make the code easier to test.

Data Transfer Objects (DTOs)

Do

- Use DTOs for all API requests and responses.

Avoid

- Returning Entity classes directly to the frontend.

Why? DTOs protect sensitive information and keep the API independent from the database structure.

Dependency Injection

Do

- Use Constructor Injection for dependencies.

Avoid

- Using field injection with @Autowired.

Why? Constructor Injection makes classes easier to test and clearly shows what a class depends on.

8.3 Python AI Services

The AI service must remain organized, reproducible, and easy to maintain.

Project Structure

Do

- Keep production code inside the app/ folder.
- Use the notebooks/ folder only for experiments.

Avoid

- Importing notebook code into the running application.

Why? Notebook code is usually experimental, while the app/ folder contains stable production code.

Code Quality

Do

- Follow Python coding standards by using Black to format code and Ruff to check for errors.
- Add type hints to public functions.

```
def predict_hours(user_id: int) -> float:  
    return 42.0
```

Machine Learning

Do

- Save the dataset version used for training.
- Record model settings (hyperparameters) and evaluation results like accuracy.

8.4 Database (PostgreSQL & Flyway)

Database Design

Do

- Give every table a primary key and use foreign keys to define relationships.
- Use plural, snake_case table names (e.g., users, timesheet_entries).
- Use singular, snake_case column names (e.g., user_id, created_at).

Database Changes

Do

- Make all database changes using Flyway migration files.
- Commit migration files together with the feature that requires them.

Avoid

- Editing the shared database manually.

Why? Flyway keeps every developer's database and the production database in sync.

8.5 Docker and AWS

Docker

Do

- Create a Dockerfile for each service and use Docker Compose to run the full application locally.
- Use multi-stage builds to reduce image size.

Avoid

- Storing passwords, API keys, or secrets inside Docker images.

AWS

Do

- Store secrets using AWS Parameter Store.
- Keep development and production resources separate.
- Use clear and consistent names for AWS resources (e.g., timesheets-dev-api, timesheets-prod-api).

9. Git, Version Control, and Issue Tracking

9.1 Core Workflow Principles

- **Isolation:** Never commit directly to the `main` or `dev` branches. All work must occur in a dedicated branch and pass through a Pull Request (PR).
- **Frequency:** Make small, frequent, and atomic commits as you complete parts of a feature. Avoid large, single commits containing unrelated changes.

9.2 Commit Message Standard

The project strictly follows the Conventional Commits format using the present-tense imperative mood (e.g., "add", not "added" or "adds").

Format: `<type>(optional-scope): short description in present tense`

Type	Use Case	Example
feat	A new feature or endpoint	<code>feat(auth): add login endpoint</code>
fix	A bug fix	<code>fix(timer): correct calculation offset</code>
chore	Maintenance, dependencies, or tool configs	<code>chore(deps): update spring boot version</code>
docs	Documentation changes only	<code>docs(readme): update installation guide</code>
refactor	Code structural changes that alter no behavior	<code>refactor(auth): move validation into service</code>
test	Adding or correcting unit/integration tests	<code>test(users): add controller unit tests</code>
build	CI/CD pipeline or build tool updates	<code>build(ci): adjust caching steps</code>

perf	Performance-specific optimizations	perf(db): add index to user email
------	------------------------------------	-----------------------------------

9.3 Branching Strategy

Branch names must reflect the operational ticket and clearly state the task.

- **Main Branches:**
 - `main`: Represents the stable production version.
 - `dev`: Integration branch where completed features are combined.
- **Working Branches:**
 - `feature/feature-name` (e.g., `feature/login-page`, `feature/timesheet-export`)
 - `fix/bug-name` (e.g., `fix/sidebar-routing`)
 - `docs/documentation-target` (e.g., `docs/update-readme`)

Branch Management Rules

- Keep branches focused on a single, isolated task.
- Delete your working branch immediately after its associated PR is successfully merged.

9.4 Pull Request Validation

Before requesting a peer review or opening a Pull Request, ensure that:

1. The codebase compiles and builds locally with zero errors.
2. All local automated unit tests pass successfully.
3. Language formatters and linters run cleanly without style violations.
4. Any outstanding merge conflicts with the target branch are fully resolved.
5. The remote CI/CD automation pipeline passes successfully.

9.5 Issue and Task Tracking Rules

Every task, bug, or feature must be represented by a clear issue tracking ticket containing these mandatory components:

- **Description:** Detail the problem, bug replication steps (actual vs. expected behavior), or feature requirements explicitly.
- **Definition of Done (DoD):** A clear, bulleted checklist of exact conditions required to consider the task complete.
- **Linkage:** Explicitly link related issues, blocks, or parent epics (e.g., "Blocked by #45", "Closes #34").
- **Scope:** Keep issues small. If a task balloons during implementation, immediately split it into clear sub-tasks with assigned owners. Do not mix unrelated changes under one issue ticket.

10. Error Handling and Input Validation

10.1 Core Principles

Do

- Catch specific exceptions rather than generic fallbacks.
- Log errors using appropriate log levels so they can be investigated.
- Return clear, non-technical, user-friendly error messages to the client.
- Validate all user input at the application boundaries before processing it.

Avoid

- Never leave empty catch blocks or silently swallow exceptions.
- Never expose raw system stack traces or database engine errors to the end user.
- Never use catch-all statements like `catch (Exception e)` or a bare `except:` unless explicitly acting as a global fallback logger.

10.2 Backend (Spring Boot)

Do

- Enforce the central `@RestControllerAdvice` pattern via `GlobalExceptionHandler.java` to handle errors consistently.
- Map distinct exception types to semantic HTTP status codes (e.g., `400 Bad Request`, `404 Not Found`).

Avoid

- Never write duplicate try-catch blocks across multiple controller methods.
- Never pass raw internal Java exception strings directly to the frontend.

Good Example

```
@RestControllerAdvice

public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)

    public ResponseEntity<String>
    handleNotFound(ResourceNotFoundException ex) {

        return
        ResponseEntity.status(HttpStatus.NOT_FOUND)

                           .body("The requested resource could not
    be found.");
    }
}
```

Bad Example

```
@PostMapping("/users")

public User createUser(...) {

    try {

        // Core logic

    } catch (Exception e) {

        return null;
    }
}
```

```
}  
  
}
```

10.3 Frontend (Angular)

Do

- Intercept operational API errors globally using an HTTP Interceptor.
- Keep component classes clean by offloading error-parsing workflows to the interceptor layer.

Avoid

- Never manage standard HTTP network errors individually within every single component file.

10.4 Python AI Service

Do

- Target specific runtime exceptions (e.g., `ValueError`, `TypeError`).
- Provide explicit operational context when re-raising custom application errors.

Avoid

- Never use a bare `except:` block combined with a `pass` statement.

Good Example

```
try:  
  
    prediction = model.predict(data)
```

```
except ValueError as error:

    raise PredictionError("Invalid prediction input.")
from error
```

Bad Example

```
try:

    prediction = model.predict(data)

except:

    pass
```

10.5 Input Validation Rules

Do

- Enforce automated validation constraints directly at the data entry points.
- Reject malformed request payloads immediately before passing data to services or repositories.

Spring Boot Early Validation Example

```
@PostMapping("/users")

public ResponseEntity<UserResponse> createUser(@Valid
@RequestBody UserRequest request) {

    return userService.createUser(request);

}
```

10.6 Pre-Commit Error Handling Checklist

Verify the following rules before staging your code:

1. No empty catch or except blocks exist in the new files.
2. All caught exceptions are logged for debugging transparency.
3. Errors returned to the client are mapped to friendly messages with no internal stack paths.
4. Input validation annotations or checks are placed on all new incoming request DTOs.

11. Logging

11.1 Logging Levels

Select log levels based on the operational importance of the event:

- **ERROR:** The system failed to complete an operation (e.g., database connection drops, caught exceptions, failed API calls).
- **WARN:** An unexpected event occurred, but the application can safely recover and continue running.
- **INFO:** Major application lifecycle milestones (e.g., application startup completion, successful user logins, finished background tasks).
- **DEBUG:** High-thorough diagnostics used exclusively by developers during local troubleshooting (disabled in production environments).

11.2 Java (Spring Boot) Implementation

Use Lombok's `@Slf4j` annotation to automatically inject a standard logger instance.

```
@Slf4j
@Service
public class UserService {
    public void login(String username) {
        log.info("User logged in successfully.");
    }
}
```

For caught exceptions, always log the error context alongside the full exception stack trace:

```
try {  
    // operational logic  
} catch (Exception e) {  
    log.error("Failed to save timesheet.", e);  
}
```

11.3 Python AI Service Implementation

Use Python's native `logging` module. Never use standard print statements for system tracking.

```
import logging  
  
logger = logging.getLogger(__name__)  
  
logger.info("AI model loaded.")  
logger.error("Prediction failed.")
```

11.4 Sensitive Data Restrictions

Never log sensitive, private, or identifying information. Even within local DEBUG logs, the following elements must never be printed to the console:

- Raw passwords or credentials
- Authentication tokens (JWTs, refresh tokens)

- Third-party API keys
- Personal identifying details (e.g., email addresses)

12. Security

12.1 Configuration and Secrets

- **Do:** Store all operational passwords, database connections, and API keys securely within environment variables (e.g., `DATABASE_URL`, `JWT_SECRET`).
- **Avoid:** Never hardcode credentials into source strings or configuration files.

12.2 Password Security

Always hash plain-text user passwords using the BCrypt encoder interface before executing a database save operation. Raw passwords must never be stored in persistent storage.

12.3 Input Validation

Validate all incoming data at the API boundary using DTO validation before it reaches the Service layer.

```
@PostMapping  
  
public ResponseEntity<?> create(@Valid @RequestBody UserRequest  
request)
```

Utilize clear built-in framework constraints such as `@NotBlank`, `@Email`, and `@Size(min = 8)` on data request objects.

12.4 Database Safety

Never construct SQL query strings manually using string concatenation, which exposes the system to SQL injection vulnerabilities. Always leverage parameterized queries, built-in repository interfaces, or Spring Data JPA.

13. AI Usage Policy

The use of AI tools such as ChatGPT, GitHub Copilot and similar coding assistants is permitted for learning, debugging, understanding concepts, generating boilerplate code, writing tests and improving documentation.

Core Ownership Rules

- You must thoroughly review, test, and understand every line of AI-generated code before executing a Git commit.
- Relying on "ChatGPT wrote it" as a defense during code evaluations or grading reviews is unacceptable. You must be able to verbally explain the logic behind your submissions.
- Ensure no private infrastructure secrets, keys, or identifying team parameters are ever pasted into public AI prompt fields.

Vibe Coding Policy

- **Vibe Coding is strictly prohibited.**
- For this project, **Vibe Coding** refers to copying or submitting AI-generated code without understanding how it works or without being able to explain it.
- Every team member is responsible for the code they contribute to the project.

Rule

If you cannot explain how your code works, you should not commit it.

14. Testing Standards

An engineering feature is not considered complete until its accompanying unit and integration tests have been successfully written and verified.

14.1 The AAA Pattern

Every unit test must follow a clear Arrange-Act-Assert structural layout:

- **Arrange:** Set up required mock data objects, preconditions, and stub behaviors.
- **Act:** Execute the specific target function or method under test.
- **Assert:** Verify that the resulting outputs or state, matches expectations.

```
@Test
void should_returnTotalHours_when_entriesExistForWeek() {

    // Arrange

    var entries = List.of(
        new TimesheetEntry(8),
        new TimesheetEntry(6)
    );

    when(repository.findByWeek(id)).thenReturn(entries);

    // Act

    var total = service.calculateWeeklyTotal(id);
```

```
// Assert  
  
assertEquals(14, total);  
  
}
```

14.2 Test Naming Structure

Test method names must use a descriptive `should_[behavior]_when_[condition]` snake_case convention. Avoid arbitrary names like `test1` or `loginTest`.

- **Good:** `should_createUser_when_requestIsValid`
- **Good:** `should_return404_when_userDoesNotExist`

14.3 Target Matrix by Framework Layer

Focus testing efforts explicitly on core system logic, covering successful flows, expected failure paths, and edge conditions. Do not waste time testing framework getters, setters, or simple generated DTO architectures.

Layer	Tool
Angular Components / Services	Jest
Angular End-to-End (E2E)	Cypress
Spring Boot Unit Testing	JUnit 5 + Mockito
Spring Boot Integration Testing	Spring Boot Test

Python AI Services

PyTest

15. Pre-Commit Verification Checklist

Before opening a feature Pull Request, confirm that you can answer yes to the following questions:

1. Have all passwords, credentials, tokens, and personal emails been kept out of console logs?
2. Are all system authentication credentials isolated in external environment variables?
3. Is all user-submitted data properly validated at the entry endpoint?
4. Are you prepared to verbally explain every line of handwritten or AI-assisted code you are submitting?
5. Have you run Spotless, Checkstyle, PMD, and SpotBugs (where applicable) and resolved all reported issues?
6. Have comprehensive tests following the AAA pattern been written for all modified logic paths?
7. Do all new test methods follow the descriptive `should_[behavior]_when_[condition]` naming rule?
8. Have you used the correct logging levels (ERROR, WARN, INFO, DEBUG) throughout the implementation?

16. Code Review and Quality Control

A Pull Request (PR) cannot be merged into shared repository branches without an approval from a teammate. Reviewers must evaluate the submission against the following criteria:

16.1 Review Criteria Checklist

- **Handbook Adherence:** Check for feature-based naming structures and formatting standards (e.g., `UserService` and `calculateWeeklyHours()` instead of `userservice` or `calc()`).
- **Test Verification:** Confirm that comprehensive tests exist covering successful outcomes, bad inputs, missing parameters, and edge cases, rather than verifying only the success paths.
- **Secret Leakage:** Ensure no raw environment strings, private variables, tokens, or security passwords are visible in source code files or runtime console log changes.
- **Error Containment:** Verify exceptions are properly handled and logged with operational context, never left empty.
- **Commit Formatting:** Check that the Git history follows the Conventional Commits present-tense standard.
- **AI Accountability:** The submission author must be capable of explaining the specific runtime intent of any code written with AI assistance.

17. Definition of Done (DoD)

A development task cannot be marked complete or closed until it passes through every sequence milestone:

Requirement	Operational Meaning
Requirements Completed	The code implementation fully solves the target acceptance criteria.
Tests Pass	All automated unit, integration, and end-to-end suites execute with zero failures.
Standards Followed	Code mirrors the structural, stylistic, and quality patterns in the handbook.
Reviewed	At least one peer developer has inspected and explicitly approved the PR.
CI Passes	The automated GitHub Actions status check finishes completely green.
Code Coverage Badges Updated	Code coverage has been generated and the project's GitHub README displays up-to-date coverage badges

	(e.g., SonarCloud or Codecov) where applicable.
Documentation Updated	Project architectural docs, structural comments, or README layouts are modified.
Verified on Public AWS Deployment	The feature has been tested successfully on the team's deployed AWS application, not only on a local machine.

18. Development and Management Templates

18.1 Pull Request Template

Summary:

[Provide a brief sentence detailing what you built, e.g., Added JWT authentication]

Related Ticket:

Closes #[Ticket Number]

Changes:

- [Itemized change log list]

Testing:

- [Specify local tests run, test frameworks used, or CI pipeline validation status]

Checklist:

- Coding standards followed
- No secret keys or environment variables committed
- Relevant documentation or templates updated
- AI-assisted code blocks fully understood by author

18.2 Bug Report Template

Summary:

[Brief description of the malfunction]

Steps to Reproduce:

1. [Step 1]
2. [Step 2]

Expected Behavior:

[What should have happened]

Actual Behavior:

[What actually happened, including raw log exceptions or HTTP error codes]

18.3 Feature Request Template

Problem:

[What limitation or user experience problem exists today?]

Proposed Solution:

[Your architectural approach or design idea]

Value:

[Why is this valuable to the user or the grading rubric?]

18.4 Technical Task Template

Task:

[Technical change item, e.g., Upgrade Angular to v21]

Goal:

[Improve application security, dependency footprint, or framework stability]

Acceptance Criteria:

- Application builds cleanly without runtime configuration failures
- Existing test suites pass with zero compatibility errors
- No deprecated libraries or packages remain in use

19. Environment Setup and Project Onboarding

Before initiating code development, verify your local developer environment contains the matching runtime tools:

- Java Development Kit (JDK) 17+
- Node.js LTS (Latest Stable Version)
- Python 3.11+
- Docker Desktop

Standard Onboarding Steps

1. Clone the central repository locally via Git.
2. Duplicate the `.env.example` template file into a new file named `.env`.
3. Fill out your local database settings within `.env`. Never track or commit the `.env` file into Git.
4. Run `docker-compose up` to initialize local PostgreSQL and infrastructure containers.
5. Verify that the backend, frontend, database, and supporting services all start successfully before beginning development.
6. Run your project's local test suites to verify framework compilation and environment health.

20. Compliance

All team members are expected to follow the standards in this handbook throughout the project.

Failure to follow these standards may result in:

- Pull Requests being rejected during peer review.
- Additional rework before code can be merged.
- Failed automated quality checks.
- Delays during integration and testing.
- Reduced contribution credit where coding standards are repeatedly ignored.

Following these standards helps keep the codebase maintainable, testable, secure and ready for demonstrations throughout the project.

21. References

- [1] Angular Team, "Angular Coding Style Guide." [Online]. Available: <https://angular.dev/style-guide>. [Accessed: Jul. 1, 2026].
- [2] Google, "TypeScript Style Guide." [Online]. Available: <https://google.github.io/styleguide/tsguide.html>. [Accessed: Jul. 2, 2026].
- [3] Prettier, "Prettier Documentation." [Online]. Available: <https://prettier.io/docs/en/>. [Accessed: Jul. 3, 2026].
- [4] Google, "Google Java Style Guide." [Online]. Available: <https://google.github.io/styleguide/javaguide.html>. [Accessed: Jul. 4, 2026].
- [5] Spring Framework, "Code Style Guide." [Online]. Available: <https://github.com/spring-projects/spring-framework/wiki/Code-Style>. [Accessed: Jul. 5, 2026].
- [6] Spring Boot, "Structuring Your Code." [Online]. Available: <https://docs.spring.io/spring-boot/reference/using/structuring-your-code.html>. [Accessed: Jul. 1, 2026].
- [7] Baeldung, "Spring Boot REST API Best Practices." [Online]. Available: <https://www.baeldung.com/rest-api-best-practices>. [Accessed: Jul. 2, 2026].
- [8] Python Software Foundation, "PEP 8 - Style Guide for Python Code." [Online]. Available: <https://peps.python.org/pep-0008/>. [Accessed: Jul. 3, 2026].
- [9] Python Software Foundation, "PEP 257 - Docstring Conventions." [Online]. Available: <https://peps.python.org/pep-0257/>. [Accessed: Jul. 4, 2026].
- [10] PyTorch, "PyTorch Documentation." [Online]. Available: <https://pytorch.org/docs/stable/index.html>. [Accessed: Jul. 5, 2026].
- [11] Scikit-learn Developers, "Developer Guide." [Online]. Available: <https://scikit-learn.org/stable/developers/index.html>. [Accessed: Jul. 1, 2026].

[12] PostgreSQL Global Development Group, "PostgreSQL Documentation." [Online]. Available: <https://www.postgresql.org/docs/current/>. [Accessed: Jul. 2, 2026].

[14] Conventional Commits Contributors, "Conventional Commits Specification v1.0.0." [Online]. Available: <https://www.conventionalcommits.org/en/v1.0.0/>. [Accessed: Jul. 4, 2026].

[15] GitHub, "GitHub Actions Documentation." [Online]. Available: <https://docs.github.com/en/actions>. [Accessed: Jul. 5, 2026].

[16] GitHub, "About Pull Request Reviews." [Online]. Available: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/reviewing-changes-in-pull-requests/about-pull-request-reviews>. [Accessed: Jul. 1, 2026].

[17] OWASP Foundation, "OWASP Top Ten Web Application Security Risks." [Online]. Available: <https://owasp.org/www-project-top-ten/>. [Accessed: Jul. 2, 2026].

[18] Spring Security, "Spring Security Reference." [Online]. Available: <https://docs.spring.io/spring-security/reference/>. [Accessed: Jul. 3, 2026].

[19] JWT.io, "Introduction to JSON Web Tokens." [Online]. Available: <https://jwt.io/introduction>. [Accessed: Jul. 4, 2026].

[20] Amazon Web Services, "AWS Systems Manager Parameter Store Documentation." [Online]. Available: <https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store.html>. [Accessed: Jul. 5, 2026].

[21] Docker, "Dockerfile Best Practices." [Online]. Available: https://docs.docker.com/develop/develop-images/dockerfile_best-practices/. [Accessed: Jul. 1, 2026].

[22] Amazon Web Services, "Amazon ECS Best Practices Guide." [Online]. Available: <https://docs.aws.amazon.com/AmazonECS/latest/bestpracticesguide/intro.html>. [Accessed: Jul. 2, 2026].

Appendices

Appendix A: Issue and Project Management Templates

Pull Request (PR) Template

Use this template whenever you create a Pull Request.

Summary

Briefly describe what this Pull Request does and why the changes were made.

Related Ticket

Closes #__

Changes

- List the main changes made.
-

Testing

- [] Unit tests added or updated
- [] Tested locally
- [] CI pipeline passes

Checklist

- [] Follows the team's coding standards
 - [] No hardcoded secrets or sensitive information
 - [] Documentation updated if needed
 - [] I understand and can explain all code, including AI-assisted code
-

Bug Report Template

Use this template when reporting a bug.

name: Bug Report

about: Report a bug or unexpected behaviour

title: "[BUG]"

```
assignees: ""  
---
```

Summary

Provide a short description of the bug.

Steps to Reproduce

1. Go to...
2. Perform...
3. Observe...

Expected Behaviour

Explain what should have happened.

Actual Behaviour

Explain what actually happened.

Screenshots or Logs

Include screenshots or error logs if available.

Additional Information

Include anything else that may help identify the problem.

Feature Request Template

Use this template when suggesting a new feature.

```
---  
name: Feature Request  
about: Suggest a new feature or improvement  
title: "[FEAT]"  
assignees: ""  
---
```

Proposed Solution

Describe the feature you would like to add.

Problem Statement

Explain the problem that this feature solves.

Value

Explain why this feature would be useful.

Additional Information

Include mock-ups, diagrams, or examples if available.

Task Template

Use this template for general development work.

```
---  
name: Task  
about: General development task  
title: "[TASK]"  
assignees: ""  
---
```

Task Description

Describe what needs to be completed.

Goal

Describe the expected outcome.

Acceptance Criteria

- [] Requirement 1
- [] Requirement 2
- [] Requirement 3

Appendix B: Conventional Commit Types

Use the following commit prefixes when writing commit messages.

Commit Type	When to Use It
-------------	----------------

feat	Adding a new feature or functionality
fix	Fixing a bug
docs	Updating documentation only
style	Formatting changes that do not affect functionality
refactor	Improving code structure without changing behaviour
test	Adding or updating tests
chore	Updating dependencies, build tools, or configuration files

Examples

feat: add login page

fix: resolve timer calculation bug

docs: update README

refactor: simplify authentication service

test: add AuthService unit tests

chore: update Angular dependencies

Appendix C: Environment Setup Checklist

Before making your first contribution, complete the following steps.

Development Tools

Install:

- Node.js (LTS version)
- Java 17 (JDK)

- Python 3.11 or later
- Docker Desktop
- Visual Studio Code

Project Setup

1. Clone the project repository.
2. Copy `.env.example` and rename it to `.env`.
3. Add your local configuration values.
4. Never commit the `.env` file.

Start Required Services

Run:

`docker-compose up`

This starts the project's required services, such as PostgreSQL.

Recommended VS Code Extensions

Install the following extensions:

- ESLint
- Prettier
- Angular Language Service
- Java Extension Pack
- Python
- Docker

Verify Your Setup

Before writing any code:

- Install project dependencies.
- Run the application.
- Run all tests.
- Make sure everything builds successfully.

Appendix D: Timesheet and AI Assistance Transparency Log

To promote transparency, each team member should record their working hours and note when AI tools were used during development.

Date	Team Member	Task or Ticket	Hours Worked	AI Tool Used	Notes
dd/mm/yyyy					
dd/mm/yyyy					
dd/mm/yyyy					

Final Pre-Pull Request Checklist

Run through this complete list before passing a ticket to a teammate for peer review:

- The code solution fulfills all feature acceptance criteria defined in the tracking issue.
- Code meets the team's precise folder layouts, naming cases, and formatting configuration parameters.
- Hardcoded runtime credentials, access keys, or plain-text credentials are completely absent from code and logs.
- Error handlers actively process and log exception flows rather than silently dropping or ignoring them.
- Automated unit or integration tests have been provided following the AAA pattern.
- Test methods use explicit `should_[behavior]_when_[condition]` naming cases.
- Commits have been cleaned up and follow the imperative present-tense Conventional Commits layout.
- Documentation, parameters, configuration rules, or API endpoints have been synchronized in the project files.
- Remote continuous integration tests pass completely without failures.
- You are fully prepared to articulate the implementation logic of all modifications, including AI-assisted segments.

Appendix E: Glossary

AAA (Arrange, Act, Assert)

A common structure for writing unit tests:

- **Arrange** : Prepare the data and objects.
- **Act** : Execute the code being tested .
- **Assert** : Verify that the result is correct.

CI/CD (Continuous Integration / Continuous Deployment)

An automated process that builds, tests, and checks code whenever changes are pushed to the repository.

DTO (Data Transfer Object)

An object used to transfer data between different parts of an application without exposing internal database entities.

Flyway

A database migration tool that automatically applies versioned SQL scripts to keep every developer's database and production database consistent.

JWT (JSON Web Token)

A secure token used to identify authenticated users when communicating with the server.

LIFT

An Angular guideline for organising project files:

- **L** : Locate files easily
- **I** : Identify the purpose of each file quickly
- **F** : Keep the folder structure as Flat as possible
- **T** : Try to be DRY (Don't Repeat Yourself)

ORM (Object-Relational Mapper)

Software that allows developers to work with database records as objects instead of writing SQL for every operation.

SRP (Single Responsibility Principle)

A software design principle stating that a class or function should have only one responsibility or one reason to change.

End of Handbook